Duale Hochschule
Baden-Württemberg
Mosbach
Lohrtalweg 10
74821 Mosbach

Portfolio Web-Programmierung

BeBetter – Automatisierte Trainingsplanerstellung

Studiengang/ -richtung:	Wirtschaftsinformatik
Kurs:	MOS- WWI25A
Gruppenmitglieder:	Andrii Kharkovshchenko - 7399391 Julian Laukert - 2529715 Alexander Geiger - 8740614
Anzahl Studierende:	3
Modul:	Web-Programmierung (Semester 3)
Prüfer:	Prof. Hartwig
Abgabedatum:	08.11.2026

Inhaltsverzeichnis

1. Abstract 3
   1.1 Deutsche Version 3
   1.2 English Version 3
2. Einführung in das Projekt 4
   2.1 Motivation und Ausgangsfrage 4
   2.2 Entscheidungen zu Projektbeginn 4
   2.3 Architekturüberblick und Vorgehen 5
3. API-Design 7
   3.1 Ressourcen und Endpunkte 7
   3.2 Zustandslosigkeit 8
   3.3 Fehlermodell 8
   3.4 Weiterentwicklung des Vertrags 9
   3.5 Was wir beim API-Design genauso und anders machen würden 9
4. Implementierung 10
   4.1 Arbeitsweise und Projektverlauf 10
   4.2 Backend und Datenpersistenz 11
   4.3 Hintergrundverarbeitung: der Wochenwechsel 12
   4.4 MPA: der Adminbereich 13
   4.5 SPA: der Nutzerbereich 15
   4.6 GraphQL: der größte Planungsfehler 17
   4.7 Integration der ExerciseDB-API 18
   4.8 Infrastruktur-Optimierung 19
   4.9 Asynchronität und Architekturgrenzen 20
   4.10 Tests: was sie zeigen und was nicht 21
5. Wissenschaftliche Reflexion 23
   5.1 Reflexion Andrii Kharkovshchenko 23
   5.2 Reflexion Julian Laukert 25
   5.3 Reflexion Alexander Geiger 26

1. Abstract

1.1 Deutsche Version

Dieses Portfolio beschreibt, wie wir den Web-Service BeBetter im Modul Web-Programmierung (Semester 3, DHBW Mosbach) zu dritt entwickelt haben und was wir dabei gelernt haben. BeBetter erstellt aus wenigen Profilangaben automatisch einen Wochentrainingsplan für das Krafttraining, egal ob man gerade erst anfängt oder schon länger trainiert. Technisch besteht der Service aus einer React-SPA für Nutzer, einer Thymeleaf-MPA für Administratoren, einem Spring-Boot-Backend mit REST- und GraphQL-Schnittstelle, einer PostgreSQL-Datenbank und der externen ExerciseDB-API. Frontend und Backend laufen als zwei getrennte Heroku-Apps.

Wie in den Vorgaben verlangt, beschreiben wir nicht das fertige Produkt, sondern unsere Entscheidungen, Fehler und Erkenntnisse auf dem Weg dorthin. Vier Erfahrungen haben uns besonders geprägt. Wir mussten lernen, mit den Grenzen einer kostenlosen Fremd-API umzugehen, eine zustandslose JWT-Anmeldung mit einer Admin-Session zu verbinden und damit zu leben, dass sich Flyway-Migrationen nachträglich nicht ändern lassen. Außerdem haben wir GraphQL zu spät eingeplant. Zu jeder dieser Erfahrungen zeigen wir, was wir entschieden haben, warum, was daraus folgte und wie wir es heute sehen. Am Ende reflektiert jedes Teammitglied einzeln seine Arbeit.

1.2 English Version

This portfolio describes how the three of us built the web service BeBetter in the Web Programming course (semester 3, DHBW Mosbach) and what we learned along the way. BeBetter automatically creates a weekly strength-training plan from a few profile details, whether someone is just starting out or has been training for a while. Technically, the service consists of a React SPA for users, a Thymeleaf MPA for administrators, a Spring Boot backend with a REST and a GraphQL interface, a PostgreSQL database and the external ExerciseDB API. Frontend and backend run as two separate Heroku apps.

As the course requires, we do not describe the finished product but our decisions, mistakes and insights along the way. Four experiences shaped the project most. We had to learn how to deal with the limits of a free third-party API, how to connect stateless JWT login with an admin session, and how to live with the fact that Flyway migrations cannot be changed afterwards. We also planned GraphQL too late. For each of these, we show what we decided, why, what followed and how we see it today. At the end, each team member reflects on their own work.

2. Einführung in das Projekt

In diesem Kapitel erklären wir, warum wir BeBetter gebaut haben, was wir am Anfang entschieden haben und wie die Architektur grob aussieht. Eine ausführliche Produktbeschreibung gibt es hier nicht. Uns geht es darum, was wir dabei gelernt haben.

2.1 Motivation und Ausgangsfrage

Die Idee BeBetter kommt von Alexander Geiger und wurde im Team gemeinsam weiterentwickelt. Ausgangspunkt war die gemeinsame Leidenschaft für Sport. Wir behandeln das Problem, das viele Einsteiger haben, wenn sie anfangen: Wer ohne Trainer Krafttraining macht, egal ob als Einsteiger oder als Fortgeschrittener, weiß oft nicht, welche Übungen zu ihm passen, wie viele Sätze und Wiederholungen sinnvoll sind und mit welchem Gewicht er starten soll. Viele Apps bieten entweder starre Standardpläne oder setzen Vorwissen voraus. BeBetter nimmt dem Nutzer diese Entscheidungen ab. Es müssen einmalig Körperdaten, Ziel, Fitnesslevel und drei Trainingstage angegeben werden und daraus erhält man einen Wochenplan mit drei Ganzkörpereinheiten, deren Übungen man nach dem Training als geschafft oder nicht geschafft markiert.

Für uns als Team hat das Thema gut gepasst, weil es die Anforderungen des Moduls fast von selbst abdeckt. Der Nutzerbereich hat viel Interaktion und eignet sich deshalb für eine SPA. Die Verwaltung ist einfach und passt zu einer MPA, die der Server rendert. Die Übungsdaten kommen von einem externen Dienst, und Profile, Pläne und Markierungen müssen dauerhaft gespeichert werden. Abbildung 1 zeigt die Startseite des Nutzerbereichs. Auf weitere Funktionen gehen wir hier nicht ein, weil es in diesem Portfolio um den Lernprozess geht.

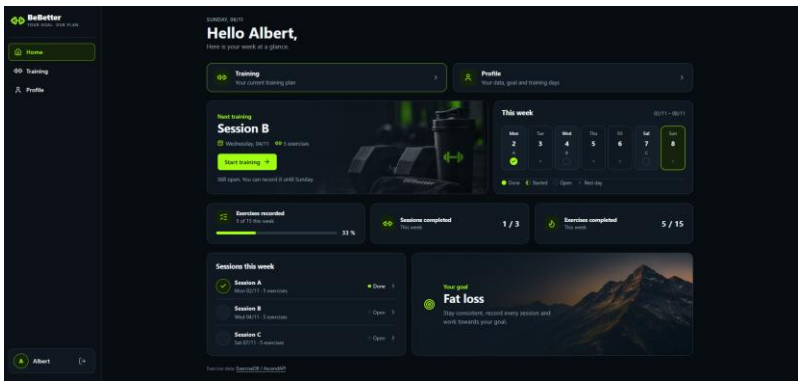


Abbildung 1: Startseite der React-SPA mit Wochenübersicht und nächster Einheit

2.2 Entscheidungen zu Projektbeginn

Am Anfang wollten wir neben dem Trainingsplan auch eine Ernährungsplanung und ein Fortschritts-Dashboard mit Streaks bauen. Nach dem ersten Feedbackgespräch mit dem Dozenten haben wir die Ernährungsplanung komplett gestrichen. Im Nachhinein war das eine der wichtigsten Entscheidungen im ganzen Projekt. Lebensmitteldaten, Nährwerte und das

Erfassen von Mahlzeiten hätten viel mehr Aufwand gemacht, ohne dass es für die Anforderungen des Moduls etwas gebracht hätte.

Bei der Priorisierung ging es uns ähnlich. In den ersten Tagen haben wir ohne klare Reihenfolge gearbeitet und wussten nicht richtig, wo wir anfangen sollen. Erst als wir die Anforderungen aufgeschrieben und nach MoSCoW in Muss, Soll und Kann eingeteilt hatten, war klar, was wichtig ist. Daraus haben wir gemeinsam einige Einschränkungen festgelegt. Es gibt drei Einheiten pro Woche an selbst gewählten Tagen, der Plan entsteht nach festen Regeln statt zufällig und lässt sich dadurch testen, und Kalender, Gast-Rolle und Demo-Modus haben wir weggelassen. Die automatische Steigerung der Folgewoche war eine Kann-Anforderung. Sie kam erst ganz am Ende, am 08.11., in einer einfachen Form dazu (Abschnitt 4.3).

Mit diesen Entscheidungen sind wir gut gefahren, weil uns mehr Zeit für die Anforderungen des Moduls blieb. Eine Schwäche haben wir aber erst spät bemerkt. Unsere Anforderungsliste war aus Sicht der Fachlichkeit geschrieben. Technische Vorgaben wie GraphQL standen zwar in der Aufgabenstellung, kamen aber am Anfang nicht als eigene Backlog-Einträge mit festem Umfang ins Backlog (siehe Abschnitt 4.6).

2.3 Architekturüberblick und Vorgehen

Abbildung 2 zeigt unser Architekturdiagramm aus der Fallstudie, aktualisiert auf den finalen Stand. Was neu ist oder sich geändert hat, ist blau markiert, was weggefallen ist, ist durchgestrichen. Neu sind vor allem GraphQL, die Hintergrundjobs und die zweite Security-Kette. Den Kalender gibt es nicht mehr, und die Steigerung der Folgewoche macht jetzt der Wochenwechsel statt eines eigenen Anpassungsservice.

Die Vorgabe war, den Service nach den Prinzipien einer „composable architecture“ bzw. MACH zu bauen. API-first sind wir, weil der Vertrag in unserer API-Dokumentation stand, bevor der Code dazu geschrieben wurde. Cloud-native sind wir, weil Frontend, Backend und Datenbank als verwaltete Dienste auf Heroku laufen. Headless sind wir, weil dasselbe Backend zwei Touchpoints bedient. Die SPA kennt weder die Datenbank noch die Fremd-API, sie spricht nur mit dem Backend. Das Backend ist damit ein Backend for Frontend (BFF). Beim „M“ für Microservices müssen wir ehrlich sein. Frontend und Backend lassen sich unabhängig deployen, das Backend selbst ist aber ein modularer Monolith. Authentifizierung, Profil, Plan und Administration liegen in eigenen Paketen, werden aber zusammen deployt. Was das für uns bedeutet hat, steht in Abschnitt 4.9.

Gearbeitet haben wir nach Scrum mit Sprints von einer Woche (Product Owner: Julian Laukert, Scrum Master: Andrii Kharkovshchenko, entwickelt haben alle drei). Die Arbeit war

gleichmäßig verteilt, im Repository hat am Ende jeder von uns zwischen 55 und 59 Commits. Der Code liegt in einem gemeinsamen Monorepo. Jede Änderung lief über einen eigenen Branch und einen Pull Request, und die GitHub-Actions-Pipeline mit Backend-Tests, Frontend-Tests, Build und statischer Prüfung musste grün sein. Ab Sprint 3 haben wir Abnahme-Checklisten im Repository geführt. Gegen Ende haben wir die aber schleifen lassen, bei Sprint 4 und 5 ist das Prüfdatum bis heute nicht eingetragen. Die Anwendung ist hier erreichbar:

Frontend: https://bebetter-frontend-8982b2eba40e.herokuapp.com
Backend: https://bebetter-fitness-app-0caa005ed4cc.herokuapp.com

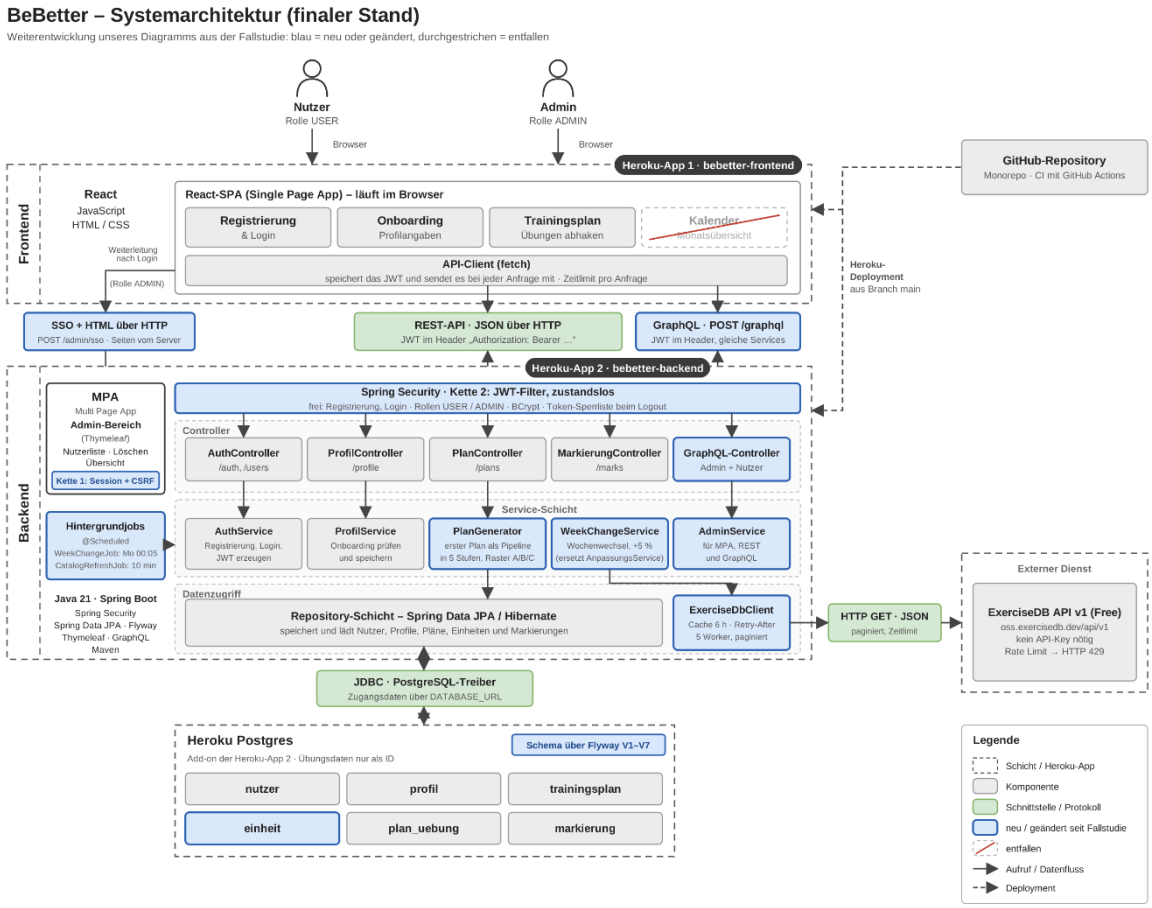


Abbildung 2: Systemarchitektur von BeBetter –Diagramm aus der Fallstudie, aktualisiert auf den finalen Stand

3. API-Design

In diesem Kapitel beschreiben wir den Vertrag unserer REST-API, die Prinzipien dahinter und wie er sich im Laufe des Projekts verändert hat. Wie wir das technisch umgesetzt haben, steht in Kapitel 4.

3.1 Ressourcen und Endpunkte

Die REST-API ist nach fachlichen Ressourcen aufgebaut. Alle Daten werden als JSON übertragen. Geschützte Ressourcen erwarten das Token, das man bei der Anmeldung bekommt, im Header `Authorization: Bearer <token>`. Den kompletten Vertrag mit Beispielen für Ein- und Ausgaben haben wir vor der Umsetzung als API-Design-Dokument festgelegt und danach im Repository als `docs/API.md` weitergepflegt.

Method	Ressource	Data In	Data Out
POST	/users	{"name", "email", "password"}	200 {"message": "User created successfully.", "userId", "name", "email", "role"} 400 {"message": "User already exists."}
POST	/auth	{"email", "password"}	200 {"token", "tokenType": "Bearer", "expiresIn": 3600, "userId", "name", "role", "profileCompleted"} 400 {"message": "Email or password is wrong."}
DELETE	/auth	Bearer <token>	200 {"message": "User logged off."}
GET	/profile	Bearer <token>	200 ProfileResponse 400 {"message": "Profile not found."}
POST, PUT	/profile	{"personal": {"age", "gender", "height", "weight"}, "training": {"goal", "fitnessLevel", "trainingDays"}}	200 {"profileId", "personal", "training", "planExists", "nextPlanFrom"} 400 {"message": "Invalid input.", "errors"}
POST	/plans	(kein Body, nur Token)	200 {"message": "Plan created successfully.", "planId", "weekStart"} 400 {"message": "Exercises could not be loaded."}
GET	/plans/current	Bearer <token>	200 {"planId", "weekStart", "weekEnd", "sessions": [{"sessionId", "label", "date", "status", "exercises": [{"planExerciseId", "name", "sets", "reps", "weight", "mark"}]}]} 400 {"message": "No plan found."}
POST	/marks	{"sessionId", "planExerciseId", "done": true}	200 {"id", "date", "done", "automatic": false} 400 {"message": "Session is not editable."}
GET	/admin/users	Bearer <token> (ADMIN)	200 [{"userId", "name", "email", "registeredAt", "role"}]
GET	/admin/stats	Bearer <token> (ADMIN)	200 {"users", "plans", "marksLastSevenDays"}
DELETE	/admin/users/{id}	Bearer <token> (ADMIN)	200 {"message": "User deleted.", "userId"} 400 {"message": "User not found."}

Tabelle 1: REST-API im Format der Vorlesung (gekürzt; für alle geschützten Ressourcen zusätzlich 400 „Token is wrong.“ bzw. „Access denied.“)

Früh entschieden haben wir, wie das Backend den Nutzer erkennt. In Ressourcen wie `/profile` oder `/plans/current` steht keine Nutzer-ID in der URL. Das Backend liest die ID aus dem geprüften Token. So kann niemand durch eine geänderte ID an fremde Daten kommen, und die Endpunkte bleiben kurz. Der Nachteil ist, dass man diese Ressourcen nur zusammen mit einem Token versteht. Das haben wir in der API-Dokumentation deshalb extra erklärt.

3.2 Zustandslosigkeit

Für die API gilt das Prinzip der Zustandslosigkeit aus dem REST-Architekturstil. Jede Anfrage bringt alles mit, was der Server zur Bearbeitung braucht. Das JWT enthält Nutzer-ID und Rolle und wird bei jeder Anfrage neu geprüft, eine Session legt der Server für die API nicht an. In der Security-Konfiguration haben wir das ausdrücklich so eingestellt:

```
@Bean
@Order(2)
SecurityFilterChain apiSecurity(HttpSecurity http, ...) throws Exception {
    http.csrf(csrf -> csrf.disable())
        .sessionManagement(session -> session
            .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(auth -> auth
            .requestMatchers(HttpMethod.POST, "/users", "/auth").permitAll()
            .requestMatchers("/admin/**").hasRole("ADMIN")
            .requestMatchers("/graphql").authenticated()
            .requestMatchers("/plans", "/plans/**", "/marks", "/profile").hasRole("USER")
            .anyRequest().authenticated())
        .oauth2ResourceServer(oauth -> oauth.jwt(...));
```

Listing 1: Zustandslose Security-Kette der API (SecurityConfig.java, gekürzt)

Beim Logout sind wir mit diesem Prinzip an eine Grenze gestoßen. Ein JWT bleibt gültig, bis es abläuft, auch wenn sich der Nutzer abmeldet. Wir haben deshalb eine kleine Sperrliste im Arbeitsspeicher gebaut, die sich die Token-ID beim Logout bis zum Ablauf merkt. An dieser Stelle hält der Server also doch wieder einen Zustand. Für eine einzelne Heroku-Instanz reicht diese einfache Lösung. Zwei Schwächen haben wir aber dokumentiert. Nach einem Neustart ist die Liste leer, und bei mehreren Instanzen hätte jede ihre eigene Liste. In einem größeren System würden wir kurze Token-Laufzeiten mit Refresh-Tokens nutzen oder eine gemeinsame Sperrliste, zum Beispiel in Redis.

3.3 Fehlermodell

Fachliche Fehler beantwortet die API immer mit HTTP 400 und einem JSON-Objekt mit `message`, bei Eingabefehlern kommt noch `errors` pro Feld dazu. Diese Konvention haben wir aus dem Referenzprojekt der Vorlesung übernommen. Für das Frontend war das praktisch, weil jede Fehlermeldung an derselben Stelle steht und direkt angezeigt werden kann. Den Nachteil haben wir erst später gemerkt. Ob ein Profil fehlt, der Zugriff verweigert wurde oder eine Eingabe ungültig ist, kann ein Client nur am Text erkennen. In unseren Sequenzdiagrammen aus der Fallstudie hatten wir für „No plan found“ sogar noch 404 vorgesehen, im Code wurde daraus 400. Bei einem neuen Projekt würden wir die Bedeutung eher über Statuscodes ausdrücken (400, 401, 403, 404, 409, 503) und die Textnachricht nur zusätzlich mitgeben.

3.4 Weiterentwicklung des Vertrags

Der Vertrag hat sich während der Entwicklung verändert. Im Vergleich zu unserem ersten API-Entwurf haben sich vor allem zwei Dinge geändert. `POST /profile` erzeugt keinen Plan mehr, dafür gibt es jetzt den eigenen Aufruf `POST /plans`. So bleibt ein gespeichertes Profil erhalten, auch wenn die ExerciseDB beim Erzeugen des Plans gerade nicht erreichbar ist. Genau das verlangt auch unsere User Story. Außerdem verweisen Markierungen jetzt auf die `planExerciseId` und nicht mehr auf die ID der Übung in der ExerciseDB. Dadurch gehört jede Markierung eindeutig zu einer Übung im Plan.

Eine dritte Änderung kam erst am 08.11. Seitdem speichert eine Markierung nur noch, ob die Übung geschafft wurde oder nicht. Gewicht und Wiederholungen haben wir aber nicht einfach aus der API gelöscht, weil ältere Clients sonst nicht mehr funktioniert hätten. `POST /marks` ignoriert die Felder jetzt, die Antwort gibt sie weiterhin als `null` bzw. `0` zurück, und im GraphQL-Schema sind sie mit `@deprecated` markiert. Die Spalten in der Datenbank bleiben ohne neue Migration bestehen. Wie man eine Schnittstelle so verkleinert, ohne bestehende Clients kaputt zu machen, kannten wir vorher nur aus der Theorie.

Alle Änderungen haben wir in `docs/API.md` nachgetragen, und das war auch richtig so. Vergessen haben wir aber, sie auch im ursprünglichen Designdokument zu ändern. Deshalb gibt es heute zwei unterschiedliche Stände.

3.5 Was wir beim API-Design genauso und anders machen würden

Beibehalten würden wir, den Vertrag früh mit konkreten JSON-Beispielen festzulegen. So hatten Frontend und Backend eine gemeinsame Grundlage für Feldnamen und Formate. Wartezeiten hat das trotzdem nicht ganz verhindert, wie Andrii und Alex in ihren Reflexionen beschreiben. Ein vereinbartes Format ersetzt eben keinen Endpunkt, der wirklich läuft. Anders machen würden wir drei Dinge. Wir würden GraphQL von Anfang an im API-Design mitplanen, statt es später anzubauen. Wir würden genauere Statuscodes verwenden. Und wir würden die Abhängigkeit von der externen API schon im Vertrag sichtbar machen, zum Beispiel mit einem eigenen Fehlerfall für „Dienst gerade nicht verfügbar“.

4. Implementierung

Dieses Kapitel ist nach den Anforderungen des Moduls aufgebaut. Zu jedem Teil beschreiben wir, was wir entschieden haben, welche Probleme und Fehler aufgetreten sind und wie wir das heute sehen. In Abschnitt 4.11 fassen wir am Ende zusammen, welche Anforderungen wir ganz, teilweise oder gar nicht erfüllt haben.

4.1 Arbeitsweise und Projektverlauf

Die Aufgaben waren grob nach Schwerpunkten verteilt. Alexander Geiger übernahm den Adminbereich als MPA, die GraphQL-Schnittstelle und das Deployment. Andrii Kharkovshchenko entwickelte die React-SPA und das Caching der ExerciseDB-Anbindung. Julian Laukert kümmerte sich um das Datenbankschema und die Migrationen, den Wochenwechsel-Job und die Release- und Live-Prüfungen. Größere Entscheidungen haben wir gemeinsam getroffen und in der Datei `docs/PLANAENDERUNGEN.md` mit Datum, Grund und Entscheider festgehalten. Tabelle 2 zeigt den Projektverlauf mit den wichtigsten Fortschritten und Rückschlägen.

Zeitpunkt	Schwerpunkt	Rückschlag oder Erkenntnis
ab 28.09.	Sprint 0: Setup, Windows-Setup, CI-Pipeline	Git, GitHub und Heroku waren für einige von uns neu
06.10.	Teamentscheidungen zum Umfang	Priorisierung nach Muss, Soll und Kann schafft erstmals ein klares Bild
bis 15.10.	Sprints 1–2: Registrierung, Anmeldung, Profil, Migrationen V1–V3	Ausgeführte Migrationen sind unveränderlich; GraphQL fehlt im Plan und wird nachgetragen
23.10.	Sprint 3: Planerstellung mit ExerciseDB, erstes Deployment auf Heroku	Tests grün, aber erste Live-Prüfung meldet HTTP 429
Sprint 4	Markierungen und Wochenwechsel-Job	Zeitabhängige Logik muss testbar gemacht werden
Sprint 5	Profil bearbeiten, Admin-MPA, GraphQL für Admin	Übergang JWT zu Session; Löschkaskade nur über neue Migration
30.10.–02.11.	Sprints 6–7: gezielte API-Prüfung, Release-Vorbereitung	Erneut HTTP 429; Client hält sich jetzt an Retry-After
06.11.	Endpräsentation; Katalog-Vorladen, GraphQL um Profil und Plan erweitert	Umfang von GraphQL zu spät gegen die Bewertungskriterien geprüft
08.11.	Steigerung der Folgewoche (+5 %), Geschlechtsfaktor beim Startgewicht, Markierung nur noch „Done“	Kann-Anforderung aus dem Puffer umgesetzt; Eingabe ohne Nutzen wieder entfernt

Tabelle 2: Projektverlauf mit Lernfortschritten und Rückschlägen

Für Code, Erklärungen und Dokumentation haben wir generative KI (Claude) genutzt. Jeder wichtige Einsatz steht im KI-Log (`docs/KI-LOG.md`, 33 Einträge) mit Datum, Werkzeug, gekürztem Prompt und wie wir das Ergebnis geprüft haben. Die passenden Stellen im Code sind mit Kommentaren wie `// KI-Log Nr. 29` gekennzeichnet. Auch beim Formulieren dieses

Portfolios hat uns KI unterstützt. Wie wir die Ergebnisse geprüft haben, beschreiben wir in den Einzelreflexionen in Kapitel 5.

4.2 Backend und Datenpersistenz

Das Backend ist klassisch in Schichten aufgebaut: Controller, Services, Regelklassen und Repositories. Die Controller nehmen nur Anfragen entgegen und geben sie weiter. Prüfregeln wie die Passwortrichtlinie oder die Grenzen im Profil liegen in eigenen Klassen, die man ohne Datenbank testen kann. Diese Aufteilung hatten wir schon in der Fallstudie mit GRASP-Mustern (Controller, Creator, Information Expert) begründet, und in der Umsetzung hat sie sich bewährt. REST, GraphQL und MPA konnten später dieselben Services nutzen, ohne dass wir Logik doppelt schreiben mussten.

Bei der Datenbank haben wir gleich am Anfang entschieden, dass Hibernate die Tabellen nicht automatisch anlegen soll. Das Schema entsteht nur über versionierte Flyway-Migrationen. Hibernate prüft beim Start nur noch, ob Entities und Tabellen zusammenpassen (`ddl-auto: validate`). Dadurch haben lokale Datenbank, Testdatenbank und Heroku Postgres immer denselben Stand, und jede Änderung lässt sich nachvollziehen.

Am meisten gelernt haben wir hier dadurch, dass sich ausgeführte Migrationen nicht mehr ändern lassen. Flyway speichert zu jeder Migration eine Prüfsumme. Ändert man eine Datei nachträglich, startet die Anwendung nicht mehr. Deshalb mussten wir Fehler mehrmals mit einer neuen Migration ausgleichen, statt sie einfach zu korrigieren. Der Check auf kleingeschriebene E-Mail-Adressen steht zum Beispiel in V3 statt in V2, weil V2 schon ausgeführt war. Am meisten beschäftigt hat uns das beim Löschen von Nutzern im Adminbereich. Die Fremdschlüssel von Profil und Trainingsplan hatten keine Löschkaskade, deshalb konnten wir ein Konto erst nach einer zusätzlichen Migration vollständig löschen.

```
-- V7__nutzer_loeschkaskade.sql
ALTER TABLE profil DROP CONSTRAINT profil_nutzer_id_fkey;
ALTER TABLE profil ADD CONSTRAINT profil_nutzer_id_fkey
    FOREIGN KEY (nutzer_id) REFERENCES nutzer(id) ON DELETE CASCADE;

ALTER TABLE trainingsplan DROP CONSTRAINT trainingsplan_nutzer_id_fkey;
ALTER TABLE trainingsplan ADD CONSTRAINT trainingsplan_nutzer_id_fkey
    FOREIGN KEY (nutzer_id) REFERENCES nutzer(id) ON DELETE CASCADE;
```

Listing 2: Nachträgliche Löschkaskade als eigene Migration

Diesen Fehler hätten wir vermeiden können. Das Akzeptanzkriterium „Ein gelöschter Nutzer verliert Konto, Profil, Pläne und Markierungen“ stand schon in unseren User Stories. Beim Entwurf der ersten Tabellen haben wir aber nur ans Anlegen von Daten gedacht. Bei einem neuen Projekt würden wir für jede Tabelle von Anfang an festlegen, was beim Löschen des übergeordneten Datensatzes passiert.

Eine weitere Herausforderung waren parallele Anfragen. Öffnet ein Nutzer die Trainingsseite in zwei Tabs gleichzeitig, könnten zwei Anfragen denselben Plan erzeugen. Die Datenbank verhindert das mit der Bedingung „ein Plan pro Nutzer und Woche“. Zusätzlich sperrt das Backend beim Erzeugen das Profil des Nutzers mit einer pessimistischen Sperre (`@Lock(PESSIMISTIC_WRITE)`). Die zweite Anfrage wartet dann und findet danach den Plan, der schon da ist.

Das Herzstück der Geschäftslogik ist das Erzeugen des ersten Plans. Das läuft in fünf Schritten ab. Zuerst laden wir Kandidaten aus der ExerciseDB und filtern sie nach Fitnesslevel und Equipment. Dann bringen wir sie nach Gerät in eine Reihenfolge und vergeben Abwechslungspunkte, +2 für einen Zielmuskel, der noch nicht dran war, und +1 für ein Gerät, das noch nicht genutzt wurde. Zum Schluss verteilen wir die Übungen auf die Einheiten A, B und C. Sätze, Wiederholungen und Startgewichte stehen als feste Tabellen in der Regelklasse `PlanRules`. Das Startgewicht passen wir zusätzlich an Körperdaten und Alter an, seit dem 08.11. auch an das Geschlecht:

```
public static BigDecimal startWeight(EquipmentCategory category, Profile profile) {
    BigDecimal base = startWeight(category, profile.getFitnessLevel());   // Tabellenwert
    if (base == null) return null;                                        // Körpergewicht
    BigDecimal bodyFactor = bmi(profile).divide(new BigDecimal("25"), 12, HALF_UP)
            .max(new BigDecimal("0.80")).min(new BigDecimal("1.20"));
    BigDecimal ageFactor = profile.getAge() >= 60 ? new BigDecimal("0.90") : BigDecimal.ONE;
    return base.multiply(bodyFactor).multiply(ageFactor)
            .multiply(genderFactor(profile.getGender()))      // m 1,2 · d 1,1 · w 1,0
            .divide(STEP, 0, FLOOR).multiply(STEP).max(STEP);   // auf 0,5 kg abrunden
}
```

Listing 3: Berechnung des Startgewichts (PlanRules.java, gekürzt)

Weil die Pipeline ohne Zufall arbeitet, kommt für dasselbe Profil immer derselbe Plan heraus, und wir können sie komplett mit Unit-Tests abdecken. In Sprint 5 hatten wir noch ausdrücklich festgehalten, dass das Geschlecht keinen Einfluss auf das Gewicht hat. Am 08.11. haben wir das im Team geändert und dokumentiert, dass eine Änderung des Geschlechts seitdem auch eine Neuplanung auslöst.

4.3 Hintergrundverarbeitung: der Wochenwechsel

Am Ende jeder Woche müssen offene Übungen ausgewertet werden, und es braucht einen neuen Plan. Dafür haben wir einen Job eingebaut, der montags um 00:05 Uhr (Europe/Berlin) läuft. Er schließt die alte Woche ab und markiert offene Übungen in angefangenen Einheiten automatisch als „nicht geschafft“. Einheiten, die gar nicht bearbeitet wurden, nimmt er in die neue Woche mit. Danach wird der Plan neu erstellt, wenn sich das Profil geändert hat, sonst wird die Vorwoche kopiert. Seit dem 08.11. gibt es dabei eine einfache Steigerung. Hat eine Einheit vier oder fünf von fünf Übungen als „Done“, steigt das Gewicht jeder Übung dieser Einheit in der nächsten Woche um 5 %, auf 0,5 kg gerundet. Aus 35,5 kg werden so 37,5 kg. Körpergewichtsübungen bekommen kein Gewicht, und nach einer Neuplanung gibt es keine

Steigerung. Ursprünglich hatten wir +5 %, gleich oder −5 % geplant. Das haben wir aber gelassen, weil es zu viele Sonderfälle gegeben hätte, die wir in der restlichen Zeit nicht mehr sauber hätten testen können.

```
@Scheduled(cron = "${training.week-change-cron:0 5 0 * * MON}", zone = "Europe/Berlin")
public void run() {
    for (Long userId : plans.findAllUserIdsWithPlan()) {
        try {
            weekChange.update(userId);
        } catch (RuntimeException ex) {
            // Ein Fehler bei einem Nutzer stoppt nicht die anderen
            LOG.warn("Week change failed for user {}", userId, ex);
        }
    }
}
```

Listing 4: Wochenwechsel-Job (WeekChangeJob.java)

Bei der Umsetzung hatten wir zwei Probleme. Zeitabhängige Logik lässt sich schlecht testen, wenn man auf den echten Montag warten muss. Deshalb haben wir die aktuelle Zeit in eine eigene Klasse `TimeProvider` gepackt, die wir in Tests auf jeden beliebigen Tag stellen können. Außerdem läuft ein geplanter Job nur, wenn der Server zu dieser Zeit auch läuft. Darauf wollten wir uns nicht verlassen. Die Methode `update` holt verpasste Wochen deshalb in einer Schleife nach, und dieselbe Logik läuft auch beim Aufruf von `GET /plans/current`. Wenn ein Lauf ausfällt, geht also nichts verloren, der Job macht den Wechsel nur schneller. So würden wir jeden Hintergrundprozess wieder bauen.

Jeder Nutzer wird in einer eigenen Transaktion verarbeitet. Geht bei einem Nutzer etwas schief, läuft der Job für die anderen trotzdem weiter. Wie wichtig das ist, haben wir bei der Neuplanung nach einer Profiländerung gesehen. Dafür braucht es die ExerciseDB, und wenn die nicht erreichbar ist, wird die Änderung für diesen Nutzer zurückgerollt und beim nächsten Lauf noch einmal versucht.

4.4 MPA: der Adminbereich

Der Adminbereich ist eine klassische Multi Page App. Spring Boot erzeugt mit Thymeleaf fertige HTML-Seiten für Übersicht, Nutzerliste und Löschbestätigung, und jede Aktion lädt eine neue Seite. Einen zweiten React-Bereich wollten wir hier nicht, weil die Verwaltung wenig Interaktion braucht und die MPA so ohne eigenen Frontend-Build auskommt. Über eine Media Query passt sich das Layout an schmale Bildschirme an. Ein kleines Skript ergänzt in der Nutzerliste eine Filterfunktion. Das Eingabefeld ist erst versteckt und wird vom Skript eingeblendet, sodass die Seite auch ohne JavaScript komplett funktioniert.

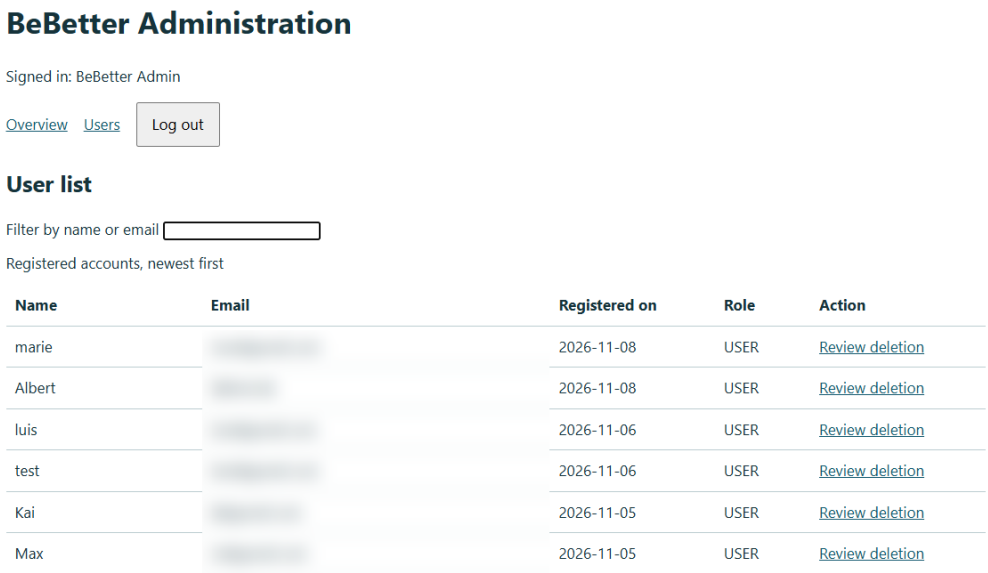


Abbildung 3: Nutzerliste der Admin-MPA mit Filterfeld (E-Mail-Adressen unkenntlich gemacht)

Schwierig wurde es erst beim Wechsel aus der SPA. Admin und Nutzer melden sich über dieselbe Login-Seite der React-App an und bekommen ein JWT. Die Thymeleaf-Seiten ruft der Browser aber direkt auf, und dabei schickt er keinen Authorization-Header mit. Wir mussten also aus dem zustandslosen Token eine normale Session machen. Das Token einfach als Parameter an die Admin-URL zu hängen, kam nicht in Frage, weil es dann im Browserverlauf und in den Server-Logs stehen würde.

```
@PostMapping("/admin/sso")
public String sso(@RequestParam String token, HttpServletRequest req,
                  HttpServletResponse res) throws IOException {
    if (!frontend.equals(req.getHeader("Origin"))) { res.sendError(403); return null; }
    Jwt jwt = decoder.decode(token);                        // Signatur und Ablauf prüfen
    User user = users.findById(Long.valueOf(jwt.getSubject())).orElse(null);
    if (user == null || !user.isAdmin()) { res.sendError(403); return null; }
    var old = req.getSession(false);
    if (old != null) old.invalidate();                      // neue Session erzwingen
    var context = SecurityContextHolder.createEmptyContext();
    context.setAuthentication(UsernamePasswordAuthenticationToken.authenticated(
        user.getEmail(), null, List.of(new SimpleGrantedAuthority("ROLE_ADMIN"))));
    sessions.saveContext(context, req, res);
    blocklist.block(jwt.getId(), jwt.getExpiresAt());     // Token nur einmal nutzbar
    return "redirect:/admin/overview";
}
```

Listing 5: Übergang vom JWT in die Admin-Session (AdminPagesController.java, gekürzt)

Nach einem erfolgreichen Admin-Login schickt die SPA das Token einmal per unsichtbarem POST-Formular an `/admin/sso`. Das Backend prüft Herkunft, Signatur und Rolle, legt eine neue Session an und sperrt das benutzte Token. Für die Adminseiten gibt es deshalb eine eigene Security-Kette mit Session und aktivem CSRF-Schutz. Gelöscht wird nur über ein bestätigtes POST-Formular mit CSRF-Token, ein GET-Aufruf zeigt nur die Sicherheitsabfrage (Abbildung 4). Das war für uns einer der lehrreichsten Teile im Projekt. Den Unterschied zwischen zustandsloser und zustandsbehafteter Anmeldung kannten wir aus der Vorlesung, richtig verstanden haben wir ihn aber erst an diesem Problem. Einen Nachteil hat die Lösung.

Die Admin-Session liegt im Speicher des Backends, nach einem Neustart muss sich der Admin neu anmelden.

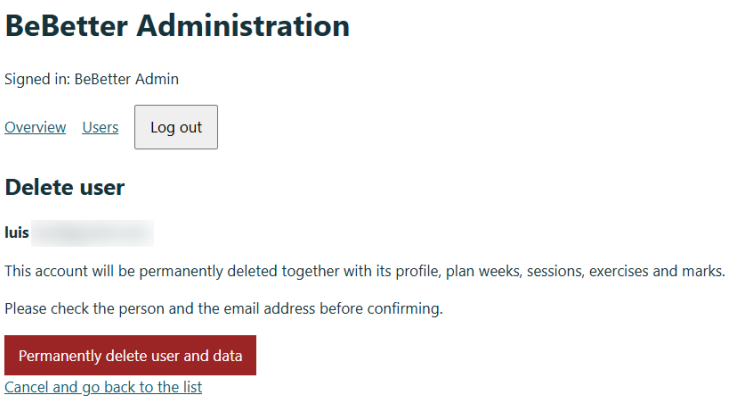


Abbildung 4: Löschbestätigung als eigene Seite; gelöscht wird erst per POST mit CSRF-Token

4.5 SPA: der Nutzerbereich

Der Nutzerbereich ist eine React-Anwendung mit React Router. Nach dem ersten Laden holt sie nur noch Daten über die API, und Seitenwechsel passieren direkt im Browser. Sie hat deutlich mehr Interaktion als die MPA, zum Beispiel ein Onboarding in mehreren Schritten mit Prüfung im Browser (Abbildung 5), die Wochenübersicht, Einheiten mit Übungsdetails und das Markieren einzelner Übungen, ohne dass die Seite neu lädt (Abbildung 6). Anmeldestatus und aktueller Plan liegen in React-Contexts, damit alle Seiten mit demselben Stand arbeiten.

```
export async function request(path, { method = 'GET', body, token,
                                      timeout = DEFAULT_TIMEOUT } = {}) {
  let response;
  try {
    response = await fetch(`${API_URL}${path}`, {
      method,
      headers: { Accept: 'application/json',
                 ...(token ? { Authorization: `Bearer ${token}` } : {}) },
      ...(body !== undefined ? { body: JSON.stringify(body) } : {}),
      signal: AbortSignal.timeout(timeout),
    });
  } catch (e) {
    if (e?.name === 'TimeoutError')
      throw new ApiError('The server took too long to respond. Please try again.', 0, null);
    throw new ApiError('Backend not reachable. Please try again.', 0, null);
  }
  return apiResponse(await response.text(), response.status, response.ok);
}
```

Listing 6: Zentraler API-Client der SPA (client.js, gekürzt)

Viel gelernt haben wir beim Umgang mit asynchronen Zuständen. Eine Anfrage kann noch laden, erfolgreich sein, fachlich scheitern oder gar nicht ankommen. Wir haben deshalb alle Anfragen über einen gemeinsamen Client laufen lassen. Er begrenzt die Wartezeit (8 Sekunden für Kontoanfragen, 60 Sekunden für die Planerstellung) und macht aus jedem Fehler ein einheitliches Fehlerobjekt. Schwierig waren auch Ladevorgänge, die gleichzeitig passieren. Wenn mehrere Komponenten den Plan fast gleichzeitig anfordern, würden sonst mehrere gleiche Anfragen rausgehen. Der Plan-Context merkt sich deshalb eine laufende

Anfrage und gibt sie an weitere Aufrufer weiter. Nach dem Speichern einer Markierung wartet er darauf und lädt danach neu. Meldet das Backend „No plan found.“, erzeugt die SPA mit `POST /plans` den ersten Plan und lädt ihn danach. Den Wechsel zwischen Konten haben wir in den Sprint-Abnahmen extra geprüft, damit keine Daten vom vorherigen Konto stehen bleiben. Den Onboarding-Entwurf speichern wir pro Nutzer-ID im Browser, damit beim Neuladen keine Eingaben verloren gehen. Passwörter und Tokens landen dort nicht. Mehr zu den Erfahrungen im Frontend schreibt Andrii Kharkovshchenko in seiner Reflexion.

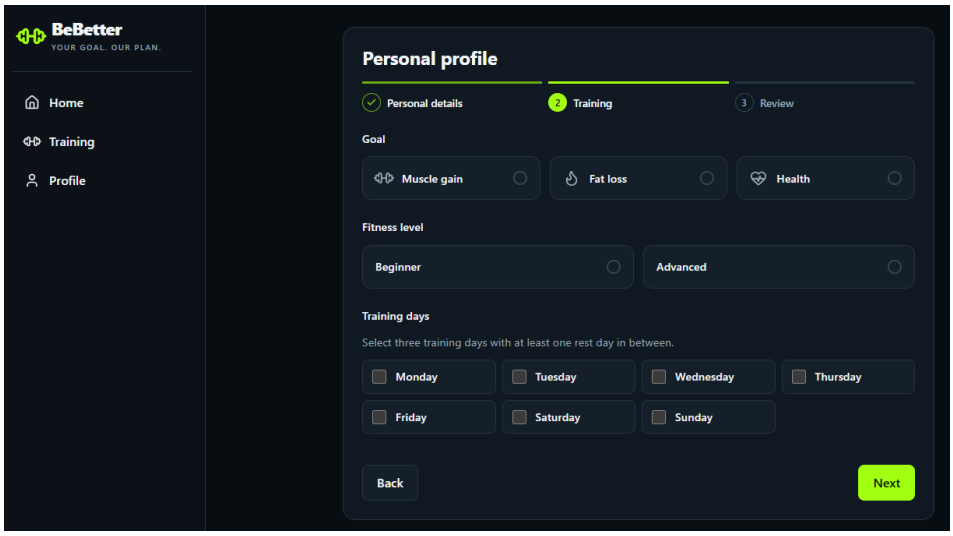


Abbildung 5: Zweiter Schritt des Onboardings mit Ziel, Fitnesslevel und Trainingstagen

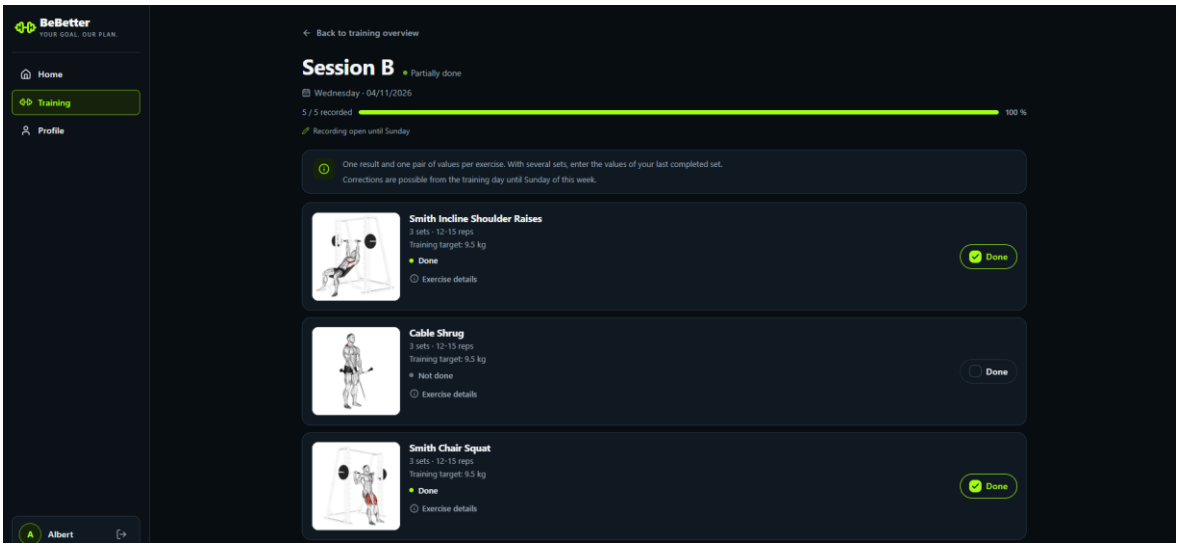


Abbildung 6: Einheit mit Markierungen per Schalter; Korrekturen sind bis Sonntag möglich

Eine Entscheidung im Nutzerbereich haben wir ganz am Ende wieder zurückgenommen. Ursprünglich haben Nutzer zu jeder Übung das tatsächliche Gewicht und die geschafften Wiederholungen eingetragen. Beim Durchgehen der Planlogik ist uns aufgefallen, dass diese Werte nirgends in die Planung eingeflossen sind. Sie haben die Bedienung also nur umständlicher gemacht. Seit dem 08.11. wird deshalb nur noch per Schalter markiert, ob eine

Übung geschafft wurde (Abbildung 6). Im Nachhinein hätten wir bei jeder Funktion früher fragen sollen, wofür ihre Daten später überhaupt gebraucht werden.

4.6 GraphQL: der größte Planungsfehler

Bei GraphQL haben wir unseren größten Fehler gemacht. Die Projektvorgabe verlangte von Anfang an, dass der Service Daten „sowohl über REST als auch GraphQL“ ausspielt. In unserem ersten Entwicklungsplan kam GraphQL trotzdem nicht vor. Am 15.10.2026 ist uns die Lücke aufgefallen, und wir haben sie in unserer Datei für Planänderungen mit dem Grund „Bewertungskriterium, fehlte im Plan“ nachgetragen. In der Projektskizze vom 19.10. wollten wir mit dem Feld `currentPlan` für den Nutzerbereich anfangen. In Sprint 5 haben wir dann aber zuerst eine Admin-API gebaut, weil wir gerade sowieso am Adminbereich gearbeitet haben und dieselben Services nutzen konnten.

```
type Query {
  adminUsers: [AdminUser!]!
  adminStats: AdminStats!
  myProfile: Profile!
  currentPlan: TrainingPlan!
}
type Mutation {
  deleteUser(id: ID!): DeletedUser
}

@Controller
@PreAuthorize("hasRole('ADMIN')")
public class AdminGraphQlController {
    @QueryMapping    public List<AdminUserDto> adminUsers() { return admin.userList(); }
    @QueryMapping    public StatsResponse adminStats()      { return admin.stats(); }
    @MutationMapping public UserDeletedResponse deleteUser(@Argument Long id) {
        return admin.delete(id);
    }
}
```

Listing 7: GraphQL-Schema (Auszug) und Admin-Resolver

Die Resolver geben nur an die vorhandene Service-Schicht weiter, deshalb war die technische Umsetzung nicht groß. Mehr Arbeit war es, das Konzept zu verstehen. Beim letzten Durchgehen der Bewertungskriterien haben wir dann ein zweites Mal gemerkt, dass wir zu kurz gedacht hatten. Bewertet werden die „Funktionalitäten verglichen mit dem REST-Server“, unsere GraphQL-Schnittstelle deckte aber nur den Adminbereich ab. Am 06.11.2026 haben wir sie deshalb um `myProfile` und `currentPlan` erweitert. Profil und Trainingsplan lassen sich jetzt auch über GraphQL abrufen. Gerade beim Plan ist GraphQL praktisch, weil ein Client aus Plan, Einheiten, Übungen und Markierungen nur die Felder abfragt, die er wirklich braucht (Listing 8). Schreibende Aktionen wie Markieren oder Profil ändern haben wir bei REST gelassen.

```
query {
  currentPlan {
    weekStart
    sessions {
      label
      date
      status
      exercises { name sets reps mark { done } }
    }
  }
}
```

Listing 8: GraphQL-Abfrage, die nur die für eine Wochenübersicht nötigen Felder lädt

Funktion	REST	GraphQL
Registrieren, An- und Abmelden	POST /users, POST/DELETE /auth	– (nur REST)
Eigenes Profil lesen	GET /profile	myProfile
Profil anlegen und ändern	POST, PUT /profile	–
Aktuellen Plan lesen	GET /plans/current	currentPlan
Plan erzeugen, Übung markieren	POST /plans, POST /marks	–
Nutzerliste und Statistik (Admin)	GET /admin/users, /admin/stats	adminUsers, adminStats
Nutzer löschen (Admin)	DELETE /admin/users/{id}	deleteUser (Mutation)

Tabelle 3: Funktionsumfang von REST und GraphQL im Vergleich

Das Problem lag also nicht an der Technik, sondern an unserer Planung. Wir wussten zwar, dass GraphQL Pflicht ist, haben aber nie festgelegt, wie viel wir umsetzen müssen und woran wir merken, dass die Anforderung erfüllt ist. Ein Backlog-Eintrag mit einem Akzeptanzkriterium wie „jede lesende REST-Funktion gibt es auch über GraphQL“ hätte das verhindert. Bei einem neuen Projekt würden wir am ersten Tag jede Bewertungsanforderung als eigenen Backlog-Eintrag mit messbarem Umfang anlegen und die API gleich für beide Schnittstellen planen.

4.7 Integration der ExerciseDB-API

Die Übungsdaten (Name, Zielmuskel, Equipment, Bild) kommen aus der kostenlosen ExerciseDB-API und werden nicht in unserer Datenbank gespeichert, im Plan steht nur die ID der Übung. Die Anbindung ist mehr als ein einzelner GET-Aufruf. Der Client lädt Übungen pro Körperteil seitenweise über einen Cursor (höchstens vier Seiten mit je 25 Einträgen), entfernt doppelte Einträge, prüft jede Antwort auf ein gültiges Format und lädt die Übungsdetails für einen ganzen Plan parallel nach. Für Verbindung und Antwort gibt es Zeitlimits von 3 bzw. 5 Sekunden. Klappt der Abruf nicht, wirft der Client eine eigene `ExerciseDbException`. Die API antwortet dann mit „Exercises could not be loaded.“, das Profil bleibt gespeichert, und die SPA bietet „Erneut versuchen“ an (Abbildung 7). Die Meldung bittet außerdem darum, nicht mehrmals hintereinander neu zu laden, weil eine Sperre des Anbieters länger als eine Minute dauern kann. Das ist auch im echten Betrieb passiert. Bei einem Test am 08.11. war die ExerciseDB erst nach etwa fünf Minuten wieder erreichbar, danach wurde der Plan ganz normal erstellt.

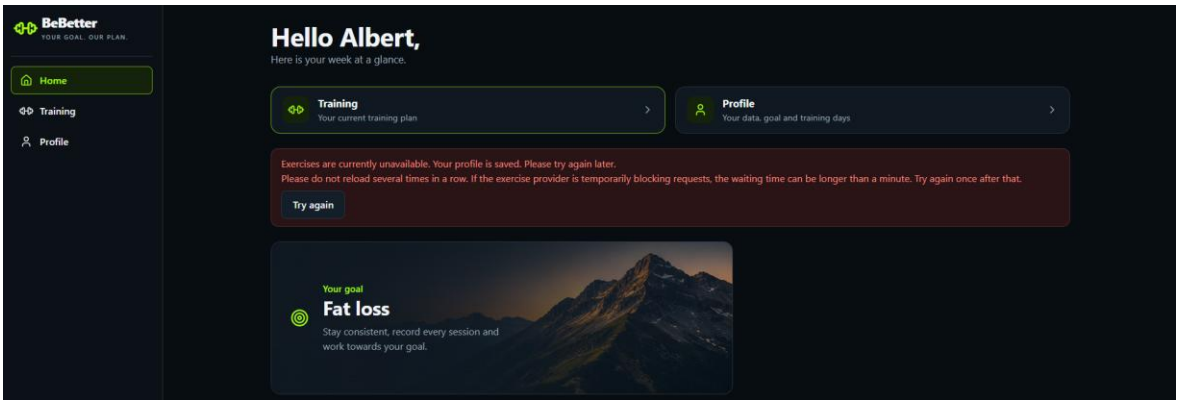


Abbildung 7: Fehlermeldung der SPA, wenn die ExerciseDB nicht erreichbar ist

Am meisten überrascht haben uns die Live-Prüfungen. Unsere automatischen Tests ersetzen die echte API durch einen Mock und waren immer grün. Die separate Live-Prüfung gegen die echte API meldete dagegen am 23.10., am 30.10. und zuletzt am 02.11.2026 um 01:47 Uhr in GitHub Actions jedes Mal HTTP 429 „Too Many Requests“. Grüne Mocktests sagen also nichts darüber aus, ob ein externer Dienst erreichbar ist. Die Live-Prüfung haben wir deshalb als eigenen Workflow behalten, den wir von Hand starten, und die Ergebnisse so dokumentiert, wie sie waren. Außerdem mussten wir akzeptieren, dass bei einer kostenlosen API der Anbieter bestimmt, wie viele Anfragen erlaubt sind. Den Client haben wir dann so erweitert, dass er sich daran hält:

```
} catch (HttpClientErrorException.TooManyRequests ex) {
    // Rate Limit wird NICHT über die Ersatzadresse umgangen
    var headers = ex.getResponseHeaders();
    blockedUntil = RetryAfterPolicy.until(
            headers == null ? null : headers.getFirst("Retry-After"),
            clock.instant());
    throw new ExerciseDbException("ExerciseDB rate limit. Please try later.", ex);
}
```

Listing 9: Reaktion auf HTTP 429 im ExerciseDbClient (gekürzt)

Bekommt der Client ein 429, wertet `RetryAfterPolicy` den Header `Retry-After` aus (als Sekunden oder als Datum) und sperrt alle weiteren Aufrufe für mindestens 60 Sekunden. Der Client kennt zwar mehrere Adressen, wechselt bei einem Rate Limit aber absichtlich nicht auf eine andere, weil wir damit nur das Limit des Anbieters umgehen würden. Beim nächsten Mal würden wir Nutzungsbedingungen und Limits schon bei der Auswahl einer API prüfen und von Anfang an überlegen, was bei einem Ausfall passieren soll.

4.8 Infrastruktur-Optimierung

Von den Bausteinen, die in der Bewertung genannt werden, nutzen wir drei. Wir haben einen Cache, besondere Datenstrukturen im BFF in Form von `ConcurrentHashMap`s und einen besonderen Autorisierungsfluss, nämlich die Übergabe vom JWT in die Admin-Session aus Abschnitt 4.4. Eine Queue und einen MCP-Server haben wir nicht eingesetzt.

Am meisten bringt der Cache im ExerciseDB-Client. Übungslisten und Übungsdetails liegen sechs Stunden lang in `ConcurrentHashMap`s. Wenn eine Liste geladen wird, füllt sich gleichzeitig der Cache für die Details, sodass beim Anzeigen eines Plans meistens kein weiterer Aufruf nötig ist. Zwischen zwei Aufrufen liegen mindestens 500 Millisekunden, damit wir die API nicht in kurzen Abständen belasten.

Nach den wiederholten 429-Antworten hat ein Cache, der erst bei der ersten Anfrage gefüllt wird, aber nicht mehr gereicht. Fiel die API genau dann aus, wenn jemand seinen ersten Plan erstellen wollte, hat die Planerstellung nicht funktioniert. Am 06.11.2026 haben wir deshalb einen zweiten Hintergrundjob eingebaut. Er lädt den Katalog direkt beim Start und erneuert alle zehn Minuten die Listen, die in weniger als 45 Minuten ablaufen würden.

```
@Scheduled(initialDelay = 0, fixedDelayString = "PT10M")
public void refresh() {
    for (String part : PlanRules.BODY_PARTS) {
        try {
            client.refreshBodyPartIfDue(part);   // nur fehlende oder bald ablaufende Listen
        } catch (ExerciseDbException ex) {
            log.warn("Katalog für {} konnte nicht aktualisiert werden: {}", part, ex.getMessage());
        }
    }
}
```

Listing 10: Hintergrundaktualisierung des Übungskatalogs (CatalogRefreshJob.java)

So lässt sich ein Plan auch während eines kurzen API-Ausfalls erstellen, solange der Cache gefüllt ist. Zur Infrastruktur gehören außerdem die beiden getrennten Security-Ketten, eine lokale PostgreSQL-Datenbank über Docker, die CI-Pipeline in GitHub Actions und ein Release-Protokoll, in dem vor jeder Veröffentlichung der geprüfte Commit und ein Datenbank-Backup festgehalten werden sollen. Eine Schwäche hat unsere Lösung aber. Cache, Sperrfrist und Token-Sperrliste liegen im Speicher eines einzelnen Prozesses. Für eine Heroku-Instanz reicht das, bei mehreren Instanzen würde es nicht mehr funktionieren. Der nächste Schritt wäre eine gemeinsame Komponente wie Redis.

4.9 Asynchronität und Architekturgrenzen

Laut Vorgabe sollen die Komponenten asynchron kommunizieren, begründete Ausnahmen sind erlaubt. Bei uns sind drei Stellen asynchron. Die SPA ruft das Backend nur über nicht blockierende `fetch`-Aufrufe mit Zeitlimit auf. Die beiden Hintergrundjobs laufen komplett unabhängig von Nutzeranfragen und kümmern sich um den Wochenwechsel und den Katalog. Und die Details zu den Übungen eines Plans laden fünf Worker-Threads parallel:

```
public Map<String, CatalogExercise> exercises(Collection<String> ids) {
    var pending = new LinkedHashMap<String, CompletableFuture<CatalogExercise>>();
    new HashSet<>(ids).forEach(id -> pending.put(id,
            CompletableFuture.supplyAsync(() -> exercise(id), workers)));
    var result = new LinkedHashMap<String, CatalogExercise>();
    pending.forEach((id, f) -> {
        try { result.put(id, f.join()); }
        catch (CompletionException ex) { /* Vorgaben bleiben ohne Details sichtbar. */ }
    });
    return result;
}
```

Listing 11: Paralleles Laden von Übungsdetails mit CompletableFuture

Synchron ist dagegen die Planerstellung selbst. `POST /plans` erzeugt den ersten Plan direkt nach dem Onboarding, und das Backend wartet dabei auf die ExerciseDB. Diese Ausnahme haben wir bewusst gemacht. Der Nutzer soll seinen Plan sofort sehen, und für die Planerstellung braucht man die Übungsdaten sofort. Durch den vorgefüllten Cache geht der Aufruf meistens schnell, und wenn etwas schiefgeht, kann der Nutzer es direkt erneut versuchen.

Aus demselben Grund haben wir keine Queue eingebaut. Eine Queue lohnt sich, wenn viele Aufträge gleichzeitig ankommen und unabhängig voneinander abgearbeitet werden können. Bei BeBetter gibt es nur wenige Hintergrundaufgaben, und die laufen sowieso zeitgesteuert. Die Anfragen an die ExerciseDB sind außerdem schon über den Thread-Pool und den Mindestabstand begrenzt. Eine Queue hätte vor allem mehr Komplexität gebracht, weil wir beim Erstellen eines Plans auf ihre Abarbeitung hätten warten müssen. Für unseren Umfang finden wir das immer noch richtig. Würden aber viele Nutzer gleichzeitig Pläne erstellen, wäre eine Queue mit späterer Rückmeldung an den Client der nächste Schritt.

Bei der Composable Architecture sehen wir das ähnlich. Die Trennung in zwei Apps und dass alles nur über die API läuft, hat gut funktioniert. Dass das Backend ein modularer Monolith ist, hat uns die Arbeit im Team leichter gemacht, weil REST, GraphQL und MPA dieselben Services nutzen konnten. Es heißt aber auch, dass ein Fehler oder Neustart im Backend alle Bereiche auf einmal trifft. Bei einem nächsten Projekt würden wir das Backend früher in eigene Services aufteilen und mit der Anbindung des Fremdservices und der Hintergrundverarbeitung anfangen.

4.10 Tests: was sie zeigen und was nicht

Getestet haben wir auf mehreren Ebenen. Im Backend gibt es 131 automatische Tests, im Frontend 49. Beide laufen bei jeder Änderung in der CI-Pipeline. In Tabelle 4 sind die Testarten aufgelistet.

Testart	Beispiele im Repository	Was geprüft wird
Unit-Test	PlanRulesTest, PasswordPolicyTest, RetryAfterPolicyTest	einzelne Regeln ohne Datenbank und Web-Stack

Testart	Beispiele im Repository	Was geprüft wird
Integrationstest	PlanIntegrationTest, TrainingIntegrationTest, UserGraphQlIntegrationTest	mehrere Schichten zusammen, mit Testdatenbank und gemockter ExerciseDB
Frontend-Test	utils/*.test.js	Validierungs- und Hilfsfunktionen
Build- und Static-Test	npm run build, npm run test:static	Produktions-Build, Auslieferung und Deep Links
Live-Prüfung	LiveExerciseDbTest (manuell gestartet)	echte Verbindung zur ExerciseDB, nicht Teil der normalen Testsuite

Tabelle 4: Testarten im Projekt

Die Tests haben uns vor allem bei den Regeln Sicherheit gegeben, zum Beispiel bei Grenzwerten der Eingaben, bei den Startgewichten und bei der Auswertung von `Retry-After`. Was sie nicht zeigen konnten, haben wir bei der ExerciseDB gemerkt. Weil die normale Testsuite die API durch einen Mock ersetzt, haben wir das Rate Limit des Anbieters erst bei der Live-Prüfung gesehen. Auch ob sich die App gut bedienen lässt und auf verschiedenen Geräten richtig aussieht, prüfen solche Tests nicht. Dafür hatten wir die manuellen Abnahmen. Diese Aufteilung würden wir beibehalten, die Live-Prüfung aber früher und öfter machen.

5. Wissenschaftliche Reflexion

In diesem Kapitel reflektiert jedes Teammitglied einzeln über die eigene Arbeit: wie strukturiert wir gearbeitet haben, wo die eigenen Stärken und Schwächen lagen, was in einem ähnlichen Projekt anders laufen würde und was wir Studierenden des nächsten Jahrgangs empfehlen.

5.1 Reflexion Andrii Kharkovshchenko

Eine Benutzeroberfläche zu sehen und sie selbst zu entwickeln, sind zwei sehr unterschiedliche Erfahrungen. Das wurde mir während der Arbeit an BeBetter deutlich. Aus Nutzersicht wirken eine Anmeldung, ein Profilformular oder die Anzeige eines Trainingsplans selbstverständlich. Bei der Umsetzung bemerkte ich jedoch, wie viele Überlegungen dahinterstehen: Was passiert nach einem Klick? Wann werden Daten geladen? Wie erfährt eine Person, dass ihre Eingabe nicht gespeichert werden konnte?

Mit dem Frontend übernahm ich einen Bereich, in dem ich bisher wenig praktische Erfahrung hatte; meine Vorkenntnisse beschränkten sich überwiegend auf Java und etwas Python. In BeBetter verwendeten wir JavaScript, JSX und CSS, React für den Aufbau der Oberfläche, React Router für die Navigation und Vite für Entwicklung und Build. Anfangs war es bereits eine Aufgabe für sich, die Zuständigkeiten dieser Technologien auseinanderzuhalten. Die Vorlesungen zur Webprogrammierung bekamen dadurch einen unmittelbaren praktischen Bezug: Inhalte aus dem Unterricht lösten neue Überlegungen zum eigenen Code aus, und umgekehrt betrachtete ich Vorlesungsinhalte durch Fragen aus der Praxis aufmerksamer.

Am stärksten musste ich mich beim Umgang mit JavaScript und React umstellen. Ich war daran gewöhnt, einen Programmablauf Schritt für Schritt zu verfolgen. Im Frontend reagieren verschiedene Bestandteile dagegen auf Eingaben, Zustandsänderungen und eintreffende Serverantworten. Insbesondere asynchrone Anfragen waren zunächst schwer nachzuvollziehen: Eine Anfrage kann bereits gestartet sein, während die benötigten Daten noch fehlen, und die Seite muss in dieser Zeit etwas Sinnvolles anzeigen. Hinzu kam die Frage, wie Informationen aktuell bleiben. Wird ein Trainingsplan verändert, muss die Anzeige den neuen Stand wiedergeben, und beim Wechsel des Benutzerkontos dürfen keine Inhalte des vorherigen Kontos zurückbleiben. Daran erkannte ich, dass eine übersichtliche Oberfläche auf einer sorgfältig organisierten Logik beruht. Eine frühzeitige Skizze der Datenflüsse hätte mir hier vermutlich geholfen.

An der Verbindung zum Backend zeigte sich, wie eng unsere Aufgaben miteinander verknüpft waren. Eine optisch fertige Seite bedeutete noch nicht, dass eine Funktion vollständig umgesetzt war; erst beim Zusammenführen unserer Ergebnisse ließ sich prüfen, ob der gesamte Ablauf funktionierte. Dabei lernte ich, Probleme genauer zu beschreiben:

Entscheidend war nicht nur, dass etwas nicht funktionierte, sondern welche Anfrage ausgelöst wurde, welche Antwort zurückkam und wie die Oberfläche darauf reagierte. Gelegentlich entstanden Wartezeiten, weil ein Arbeitsschritt auf einem anderen aufbaute. Daraus nehme ich mit, dass Aufgaben nicht nur nach Zuständigkeiten, sondern auch nach ihren gegenseitigen Abhängigkeiten geplant werden sollten.

Zu meiner fachlichen Verantwortung kam die Rolle des Scrum Masters hinzu. Ich musste also meine eigenen technischen Aufgaben bewältigen und gleichzeitig unsere Arbeitsorganisation im Blick behalten. Es ging darum, Hindernisse sichtbar zu machen, Absprachen zu unterstützen und darauf zu achten, dass unsere nächsten Schritte für alle nachvollziehbar waren; die Verantwortung für Entscheidungen blieb dabei beim gesamten Team. Unsere Zusammenarbeit empfand ich als eine wesentliche Stärke des Projekts. Wir konnten offen ansprechen, wenn jemand nicht weiterkam, und hatten Verständnis dafür, dass die Einarbeitung unterschiedlich viel Zeit benötigte. Auch Begriffe wie Commit, Merge und Deployment erhielten erst durch die praktische Arbeit mit Git und Heroku eine konkrete Bedeutung.

Beim Einsatz von KI veränderten sich unsere Bedürfnisse im Verlauf der Arbeit. Während des Brainstormings waren vor allem Anregungen hilfreich, bei der Umsetzung standen Verständnisfragen, Fehlermeldungen und Informationen zu Bibliotheken im Vordergrund. Besonders nützlich war es, mir ungewohnte React-Konzepte anhand eines konkreten Problems erläutern zu lassen. Ein Vorschlag war für mich aber nicht allein deshalb brauchbar, weil er überzeugend wirkte. Ich musste nachvollziehen können, welche Aufgabe der Code erfüllt und ob er zu unserer Anwendung passt. Eine Erklärung kann den Einstieg erleichtern; das Verständnis entsteht jedoch erst, wenn man sich selbst mit den Zusammenhängen beschäftigt.

Für ein nächstes Projekt würde ich früher einen kleinen Ablauf vollständig umsetzen, von einer Eingabe im Frontend über die Verarbeitung im Backend bis zur Rückmeldung, um Unklarheiten zu erkennen, bevor weitere Funktionen darauf aufbauen. Wiederverwendbare Komponenten, einheitliche Fehlermeldungen, die Prüfung von Eingaben und Tests würde ich von Anfang an berücksichtigen und in der Sprintplanung kleinere Arbeitsschritte vorsehen.

Studierenden des nächsten Jahrgangs würde ich empfehlen, möglichst früh einen vollständigen Ablauf vom Frontend bis zum Backend umzusetzen, auch wenn er zunächst nur aus einer einzigen Funktion besteht. So zeigt sich rechtzeitig, ob Schnittstellen und Datenformate wirklich zusammenpassen. Außerdem sollten Aufgaben so geplant werden, dass niemand lange auf einen anderen warten muss, und Fragen offen im Team

angesprochen werden, statt allein an einem Problem festzuhängen. KI kann dabei eine gute Unterstützung sein, ersetzt aber nicht die eigene Auseinandersetzung mit dem Code.

Mein persönlicher Fortschritt lässt sich deshalb nicht nur an der fertigen Oberfläche festmachen. Ich kann heute besser einschätzen, welche Überlegungen hinter einer funktionierenden Webanwendung stehen und an welchen Stellen ich mein Wissen noch vertiefen muss. Gleichzeitig habe ich erfahren, dass man auch als Anfänger Verantwortung übernehmen kann, wenn man bereit ist, Fragen zu stellen und sich mit Schwierigkeiten auseinanderzusetzen.

5.2 Reflexion Julian Laukert

Die Idee, eine eigene App zu entwickeln, fand ich von Anfang an sehr spannend, genauso wie den gesamten Weg dorthin: alles wie ein richtiges Projektteam zu planen, es anschließend umzusetzen und am Ende ein echtes Produkt entwickelt zu haben. Für mich war das eine wertvolle und lehrreiche Erfahrung, aus der ich viel für das spätere Arbeitsleben mitnehme, egal ob es dort um eine App oder um ganz andere Aufgaben geht. Das Vorgehen zu planen, Zwischenstände zu kontrollieren, sich abzusprechen und auch mit dem Stress umzugehen, der in so einem Projekt entstehen kann, hat mir viel für das nächste Mal beigebracht.

Als Team haben wir sehr eng zusammengearbeitet und unsere Sprints mit klar verteilten Aufgaben abgearbeitet. Ich habe hauptsächlich im Backend gearbeitet. Neben häufigen Online-Meetings haben wir oft nach den Vorlesungen gemeinsam in der Bibliothek weitergearbeitet, und unseren Zeitplan haben wir insgesamt gut eingehalten. Offiziell war ich außerdem Product Owner; in unserem Dreierteam haben wir die Rollen aber nicht streng getrennt und wichtige Entscheidungen gemeinsam getroffen.

Die größte Herausforderung bestand für mich darin, bei all den Funktionen den Überblick zu behalten. Ich hatte vorher noch nie eine App entwickelt, und zu verstehen, wie Flyway, der Scheduler, Thymeleaf, Spring Security und GraphQL zusammenhängen, war anfangs nicht einfach. Im Laufe des Projekts habe ich mich hier stark verbessert. Mittlerweile kann ich mir viel besser vorstellen, wie eine Webanwendung aufgebaut ist und warum bestimmte Teile so umgesetzt werden müssen.

Nicht alles ist gut gelaufen. Unser größter Fehler war, dass wir GraphQL zu spät eingeplant haben, obwohl es von Anfang an in den Anforderungen stand. Erst beim abschließenden Prüfen der Bewertungskriterien ist uns aufgefallen, dass unsere Schnittstelle im Vergleich zu REST zu wenig abdeckte. Da ich für das Backlog verantwortlich war, sehe ich hier auch meinen eigenen Anteil. Außerdem hätten wir in der mittleren Projektphase das Tempo höher halten können; dadurch wurde die Schlussphase anstrengender als nötig.

Wichtig war für mich auch die Frage, wie und in welchem Umfang man KI richtig nutzt. Ich habe mir für neue Technologien Schritt-für-Schritt-Anleitungen erstellen lassen, sie abgearbeitet und so lange nachgefragt, bis ich verstanden habe, warum etwas so funktioniert. Dabei habe ich gelernt, dass funktionierender Code noch kein verstandener Code ist. Ob eine Lösung wirklich zu unserem System passt, habe ich deshalb über Tests und Live-Prüfungen kontrolliert, und die KI-Stellen im Code haben wir mit Verweisen auf unser KI-Log gekennzeichnet. Insgesamt habe ich viel besser gelernt, KI so einzusetzen, dass ich an den richtigen Stellen Zeit spare und Dinge vereinfache, aber trotzdem verstehe, wo ich gerade stehe.

Am meisten Spaß gemacht hat mir die Zusammenarbeit im Team. Wir hatten eine sehr gute Harmonie, konnten offen sagen, wenn jemand nicht weiterkam, und haben uns auch in der stressigen Schlussphase nicht aus der Ruhe bringen lassen. In einer solchen Konstellation habe ich mit diesem Projekt zum ersten Mal gearbeitet. Es hat mir außerdem gezeigt, was man alles selbst entwickeln kann, und ich habe mir fest vorgenommen, in Zukunft eigene Anwendungen für mich zu entwickeln.

Bei einem nächsten Projekt würde ich die Zusammenarbeit im Team genauso gestalten, die Herangehensweise aber noch gründlicher planen: alle Anforderungen gleich zu Beginn mit klarem Umfang festhalten, sie regelmäßig kontrollieren und immer wieder den Gesamtzusammenhang prüfen, um sicher zu sein, dass wir auf dem richtigen Weg sind. Zukünftigen Gruppen empfehle ich, sich ihr Team genau auszusuchen und die Aufgaben noch klarer zu verteilen.

Abschließend möchte ich mich bei Alex und Andrii für die Zusammenarbeit an diesem Projekt bedanken. Wir sind ein eingeschweißtes Team geworden, konnten uns immer aufeinander verlassen, und ohne die beiden hätte mir das Ganze nur halb so viel Spaß gemacht. Ich bin stolz auf das, was ich in dieser Zeit gelernt habe und was wir als Team geleistet haben, und freue mich auf zukünftige Projekte.

5.3 Reflexion Alexander Geiger

Das Projekt BeBetter war für mich eine besonders interessante und lehrreiche Erfahrung. Zu Beginn stand vor allem die technische Umsetzung der Anforderungen im Vordergrund. Im Laufe des Projekts habe ich jedoch gemerkt, dass es um deutlich mehr geht, als nur vorgegebene Aufgaben zu erledigen. Besonders spannend fand ich es, eine eigene Idee zu entwickeln und daraus Schritt für Schritt ein vollständiges Produkt entstehen zu lassen. Die ursprüngliche Projektidee kam von mir; gemeinsam haben wir sie im Team weiterentwickelt und um zusätzliche Ideen ergänzt.

Innerhalb unseres Teams waren die Aufgaben klar verteilt. Ein Teammitglied kümmerte sich hauptsächlich um das Frontend, ein weiteres um die Datenbank und Infrastruktur, und ich übernahm den Backend-Bereich, das Deployment und teilweise die Koordination des Teams. Die wichtigen Entscheidungen haben wir gemeinsam getroffen. Rückblickend hat diese Aufteilung gut funktioniert, weil jeder seine Stärken einbringen konnte. Auch wenn sich die Anzahl der Commits unterschieden hat, war die tatsächliche Arbeitsbelastung insgesamt recht gleichmäßig verteilt.

Eine der größten Herausforderungen war es zu Beginn, sich in die verschiedenen Technologien einzuarbeiten. GitHub und Heroku waren für einige von uns neu, und Begriffe wie REST, GraphQL und APIs waren zunächst eher theoretisch. Während der Entwicklung habe ich zunehmend verstanden, welchen Zweck die einzelnen Technologien erfüllen und wie sie miteinander kommunizieren. Gerade dieser Lernprozess war für mich ein wichtiger Bestandteil des Projekts.

Ich habe überwiegend selbstständig im Backend gearbeitet. In regelmäßigen Besprechungen haben wir den aktuellen Stand ausgetauscht und unsere unabhängig entwickelten Bestandteile zusammengeführt. Wir haben mit Sprints und wöchentlichen Zielen gearbeitet und unsere Aufgaben anhand des Backlogs priorisiert. Unser Zeitplan hat insgesamt gut funktioniert, und auch die Zusammenarbeit über GitHub lief fast reibungslos; während des gesamten Projekts hatten wir nur einen größeren Merge-Konflikt. Trotzdem gab es Punkte, die wir besser hätten organisieren können. Teilweise musste ein Teammitglied auf die Fertigstellung eines anderen Bereichs warten, bevor es selbst weiterarbeiten konnte. Außerdem hat unsere Architektur zwar eine sinnvolle Trennung der Verantwortlichkeiten ermöglicht, wir haben aber nicht alle Bereiche als vollständig unabhängige Services umgesetzt. Bei einem erneuten Projekt würde ich genauer überlegen, welche Komponenten tatsächlich voneinander getrennt werden sollten.

Eine weitere Herausforderung war der sinnvolle Einsatz von KI. Wir haben KI intensiv genutzt und dadurch an vielen Stellen Zeit gespart, mussten aber erst lernen, die Werkzeuge richtig einzusetzen. Gerade am Anfang entstand dadurch teilweise unnötiger Aufwand. Mit der Zeit habe ich gelernt, Aufgaben gezielter zu formulieren, die Ergebnisse kritisch zu prüfen und generierten Code nicht einfach ungeprüft zu übernehmen. Mir war wichtig, den Code zu verstehen, damit die KI eine Unterstützung bleibt und den eigentlichen Lernprozess nicht ersetzt.

Besonders motiviert hat mich, wie schnell aus einer zunächst kleinen Idee ein umfangreiches Softwareprojekt entstanden ist. Es war meine erste Erfahrung mit einem Projekt dieser Größenordnung, und zu sehen, wie aus einzelnen Codebestandteilen eine tatsächlich

nutzbare Website wird, hat mein Interesse an der Softwareentwicklung deutlich verstärkt. Ich habe dadurch sogar ein neues Hobby für mich entdeckt und möchte künftig weitere eigene Projekte umsetzen. Fachlich konnte ich vor allem meine Kenntnisse mit Spring Boot, REST und GraphQL erweitern. Gleichzeitig habe ich gemerkt, dass ich mich zu Beginn teilweise zu sehr auf die technischen Anforderungen konzentriert habe, ohne den übergeordneten Zweck zu betrachten. Gute Software muss nicht nur technisch funktionieren, sondern auch einen nachvollziehbaren Nutzen für ihre Zielgruppe bieten.

Bei einem erneuten Projekt würde ich wieder eine ähnliche Technologieauswahl treffen und die Aufgabenverteilung beibehalten. Allerdings würde ich von Anfang an mehr Zeit für Tests und Dokumentation einplanen und die Integration der einzelnen Komponenten früher und parallel zur Entwicklung vorantreiben. Persönlich möchte ich mich beim nächsten Mal stärker mit dem Frontend beschäftigen. Zukünftigen Projektteams würde ich empfehlen, die Aufgaben früh nach Stärken zu verteilen, realistische Zeitpläne aufzustellen, regelmäßig miteinander zu kommunizieren und zunächst ein kleines, funktionierendes Grundgerüst aufzubauen, bevor weitere Funktionen ergänzt werden. Insgesamt bin ich mit dem Ergebnis und meiner persönlichen Entwicklung sehr zufrieden. Besonders wertvoll war für mich, Verantwortung für einen eigenen Bereich zu übernehmen und gleichzeitig den Überblick über das Gesamtprojekt zu behalten.